

FACULDADE DE INFORMÁTICA E ADMINISTRAÇÃO PAULISTA (FIAP),

Interconectividade de Redes e Hiperconvergência

SÃO PAULO – SP

2024

BACHARELADO EM ENGENHARIA DA COMPUTAÇÃO

Interconectividade de Redes e Hiperconvergência

Vitor de Andrade Ferreira Dias – RM 957132

Guilherme Macario da Silva - RM: 84057

Kaique Carvalho da Silva - RM: 93040

Julio Cesar de Andrade Santana – RM 88100

BENJAMIM LUCAS RODRIGUES GONÇALO - RM96026

Projetos que estamos trabalhando são:

Mariana

Análise de Dados Mariana A falta de uma visão unificada do histórico de compras e demanda de serviços leva à extração manual de informações, consumindo tempo significativo. É necessário desenvolver uma ferramenta que automatize a extração e análise de dados com critérios pré-estabelecidos, proporcionando informações de maneira visual e de fácil compreensão.

Raísa Barbosa

Novo Projeto: Controle de Mochilas e Periféricos Controle de Compras de Malas e acessórios para força de vendas, para acompanhar o que cada colaborador tem solicitado e com que frequência a troca ocorre. Isso poderá mensurar qualidade dos materiais, auxiliar no controle de trocas, tratativa dos fornecedores e auxiliar no planejamento do budget por Centro de Custo.

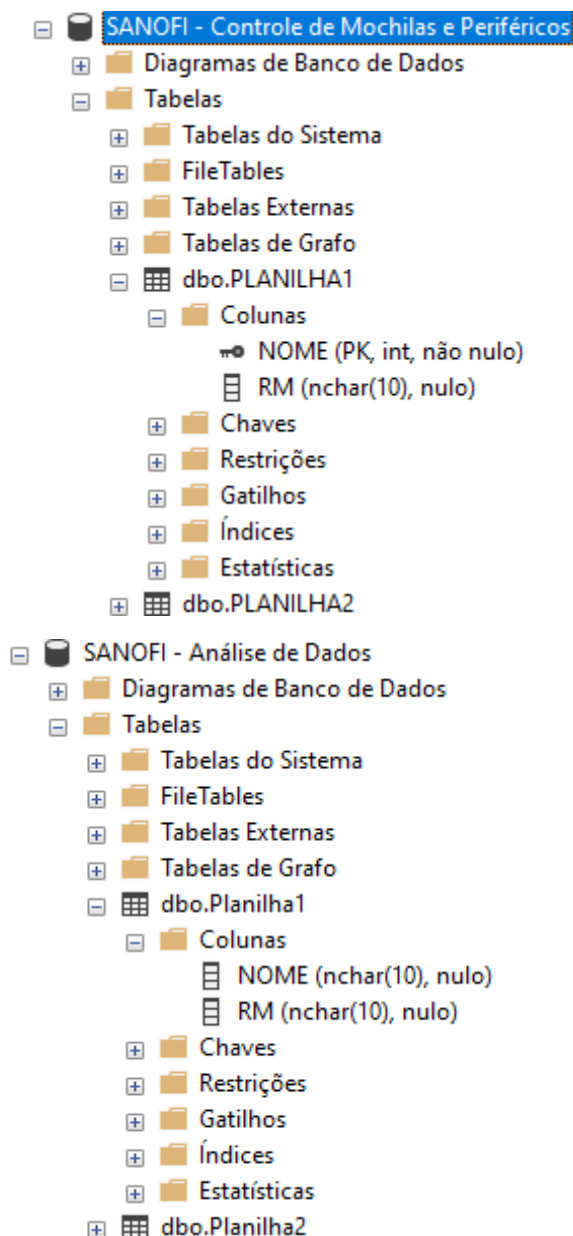
SOLUÇÃO

Descrição dos objetivos e do escopo (fronteiras; até onde vai) do seu projeto.

O nosso projeto consiste em unificar todas as planilhas que a Sanofi utiliza e fazer uma só em obdc, que atenda todas as necessidades deles.

Por trás dessa planilha existem muitas coisas interligadas que vão fazê-la acontecer.

Unificar todas as planilhas criando um banco de dados no SQL, seguindo o exemplo a seguir que é fictício.



Nesse caso fizemos dois bancos de dados simulando duas planilhas da Sanofi, estamos esperando eles enviarem as planilhas reais para inserir as colunas reais no banco de dados, porém isso não descarta nossa ideia, nossa ideia é manter todas as planilhas dentro de um banco de dados só, de acordo com cada projeto, depois fazer um inner Join em todas as planilhas e trazer a informação consolidada para UMA planilha só.

Nosso projeto tem três passos.

Primeiro buscar de onde vem as informações da Sanofi.

Sabemos que eles extraem as informações do SAP, o SAP possui uma função que é o SAP SCRIPTING, esse SAP SCRIPT já é feito para automatizar as tarefas de rotina do SAP.

Podemos pegar esse script do SAP e transformá-lo de código de python, é um processo simples, porém exige nível de conhecimento de programação.

Transformação se dá necessário porque vamos utilizado como um processo de RPA, para extrair de forma automática os dados de lá.

Iremos fazer um processo de ETL, com knime ou pentaho, para jogar esses dados que foram extraídos do SAP para o nosso banco de dados do SQL.

Após extraídos os dados do e transformados através do processo de ETL ao SQL, vamos conectar o Excel no SQL e suprir as necessidades das planilhas e consolidar em uma só para Sanofi.

TECNOLOGIAS:

EXCEL:



Atualizar todos em tempo real através de obdc.

SAP e SAP SCRIPTING:



Verificar se conseguimos extrair informações da fonte de dados deles que é o SAP

KNIME:



Utilizamos para caráter de exemplo, o knime para ETL, mas podemos utilizar o pentaho também.

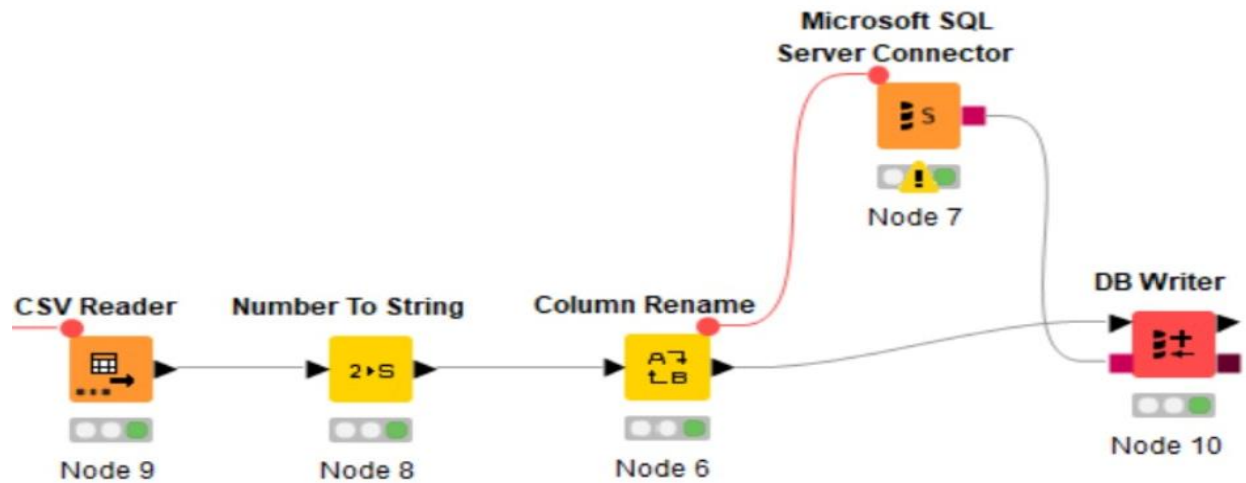
Python:



Para extrair informações do SAP, automatizar processos e também para agendar automações através do agendador de tarefas do Windows ou dentro do servidor de SQL.

Futuramente pensamos em utilizar aplicações web, API para extrair informações de sites da intranet da Sanofi.

➤ **EXEMPLO DE ETL:**



Node 9, lê a planilha extraída do SAP.

Node 8, converte os dados da planilha.

Node 6 renomeia as colunas.

Node 7 Conecta no SQL.

Node 10 Escreve no banco de dados.

EXEMPLO REAL E FUNCIONAL DE CÓDIGO DE EXTRAÇÃO DE INFORMAÇÃO DO SAP

```

def abrir_sap():
    path = r"C:\\Program Files (x86)\\SAP\\FrontEnd\\SapGui\\saplogon.exe"
    subprocess.Popen(path)
    time.sleep(5)

def conectar_ao_sap():
    try:
        SapGuiAuto = win32com.client.GetObject('SAPGUI')
        if not type(SapGuiAuto) == win32com.client.CDispatch:
            return None

        time.sleep(4)
        print("Sap gui encontrado")

        application = SapGuiAuto.GetScriptingEngine()
        if not type(application) == win32com.client.CDispatch:
            SapGuiAuto = None
            return None

        time.sleep(5)
        print("Application presente")
        time.sleep(5)

        connection = application.OpenConnection("TRANSLAÇÃO QUE A SANOFI IRÁ UTILIZAR", True)
        time.sleep(5)

        if not type(connection) == win32com.client.CDispatch:
            application = None
            SapGuiAuto = None
            return None

        session = connection.Children(0)
        if not type(session) == win32com.client.CDispatch:
            connection = None
            application = None
            SapGuiAuto = None
            return None

        print("Session presente")
        time.sleep(6)

        return session

    except Exception as e:
        print(f"Erro ao conectar ao SAP: {e}")
        return None

```

EXEMPLO REAL E FUNCIONAL DE CÓDIGO DE EXTRAÇÃO DE INFORMAÇÃO DO SAP

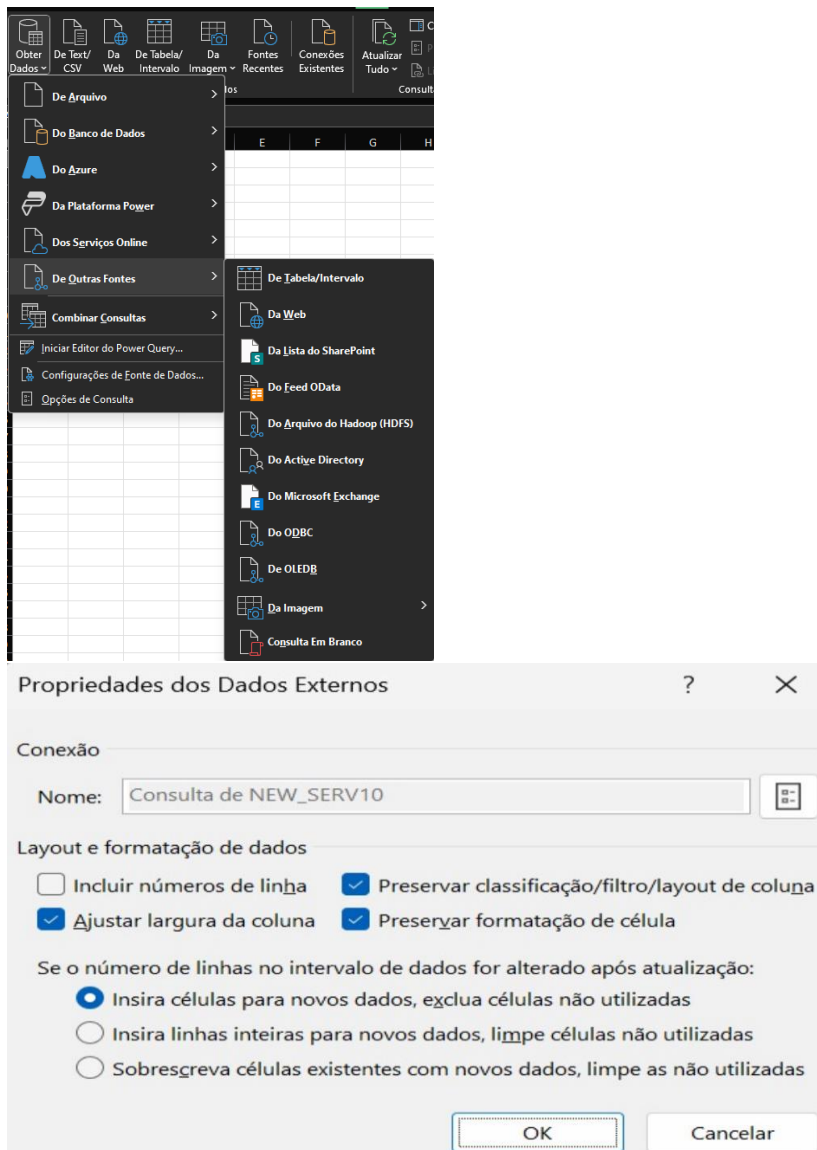

```
def exportar_dados(session, ultimo_mes_str, ultimo_ano, iteracao):
    try:
        session.findById("wnd[0]/usr/txtP_SYEAR").text = ultimo_ano
        session.findById("wnd[0]/usr/ctxtS_SPERIO-LOW").text = ultimo_mes_str
        session.findById("wnd[0]/usr/ctxtS_SPERIO-HIGH").text = ultimo_mes_str
        time.sleep(6)

        os.system(r"C:\GUILHERME\CARS\Scripts\CARS\EXPO2.VBS")
        session.findById("wnd[1]/usr/ctxtDY_PATH").text = "CAMINHO PARA SALVAR O ARQUIVO EXCEL"
        session.findById("wnd[1]/usr/ctxtDY_FILENAME").text = f"CARS_EXPO{iteracao}.xlsx"
        time.sleep(6)
    |
    return True
except Exception as e:
    print(f"Erro ao exportar dados: {e}")
    return False
```

Lembrando que o código acima ele não está com as bibliotecas que iremos utilizar (importar), quando ele estiver pronto e funcional vamos introduzir eles.

Também vamos contextualizar e padronizar o programa de acordo com as normas UML

EXCEL ODBC:



ESSE DOIS PASSOS SÃO OPÇÕES DO EXCEL, ELE CONECTA FONTE DE DADOS DO SQL NO EXCEL.

Propriedades da Conexão

Nome da conexão: Consulta de NEW_SERV10

Descrição:

Uso Definição Usado Em

Tipo de conexão: Consulta de Banco de Dados

Arquivo de conexão: Procurar...

☐ Sempre usar arquivo de conexão

Cadeia de conexão:

☐ Salvar senha

Tipo de comando: SQL

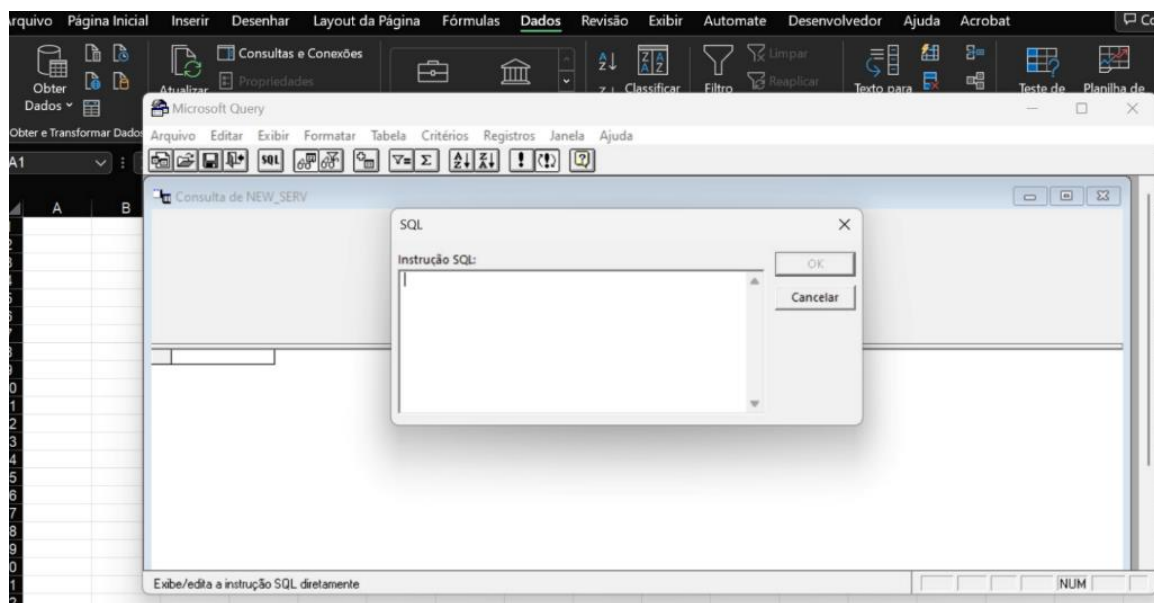
Texto do comando:

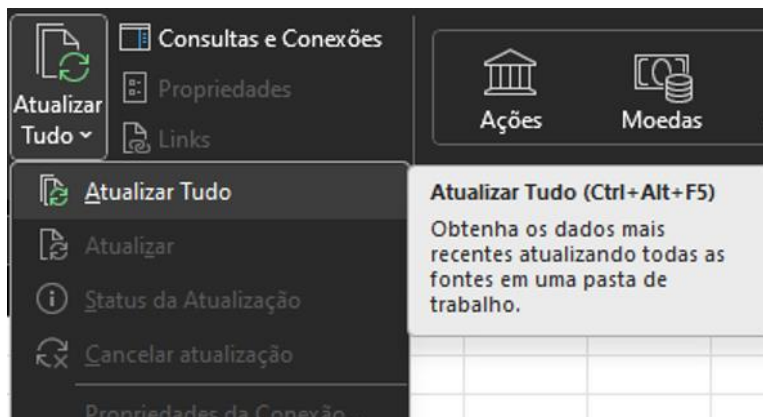
Serviços do Excel: Configurações de Autenticação...

Editar Consulta... Parâmetros... Exportar Arquivo de Conexão...

OK Cancelar

NESSE IMAGEM ACIMA PODEMOS COLOCAR A QUERY DO SQL COM INNER JOIN EM OUTRAS TABELAS QUE JÁ VÃO ESTAR NO NOSSO BANCO DE DADOS.





A PARTE DA ATUALIZAÇÃO DA PLANILHA PODE SER ESCOLHIDA PELO USUÁRIO, SEJA QUANDO ABRIR A PLANILHA, SEJA EM 5 EM 5 MINUTOS, COMO FOR A NECESSIDADE DELE E CLARO NÃO DEVEMOS ESQUECER QUE O BANCO DE DADOS DEVE ESTAR ATUALIZADO E AUTOMATIZADO.

CONCLUSÃO

Hoje nós ainda estamos à mercê de mentorias com a Sanofi.

Sabemos que a nossa ideia pode suprir mais de um projeto, queremos dar uma solução para Sanofi que não torne um problema para o usuário final, queremos manter o que eles têm mas implementar tecnologia no back-end deles, eles vão continuar utilizando as planilhas, porém vamos fazer o trabalho de bastidores deles para tornar tudo mais produtivo e mais modernizado e assim aumentar a efetividade deles.

Até outubro queremos colocar as ideias que temos hoje em prática e funcional, a partir de setembro vamos começar a utilizar inteligência artificial na nossa solução para análise preventiva de dados e também vamos entender as necessidades dele e como utilizar a inteligência artificial para tornar nosso projeto mais assertivo e mais funcional.

Também estamos pensando em fazer uma aplicação web para melhorar o sistema deles, que hoje é muito engessado, devido a rotina deles.

Por enquanto temos muito a trabalhar na ideia que temos e muito a desenvolver e descobrir conforme formos fazendo as mentorias.